代理对接文档

兑换接口供代理的服务器调用，用于查询卡密、核对买家账号、提交兑换和查询订单结果。买家在你自己的页面上操作，不需要访问我们的网站。

接口地址	认证
`https://recharge.chongplus.store/api/open/v1`	`Authorization: Bearer <API 密钥>`
格式	版本
HTTPS，JSON（UTF-8），服务器之间调用	v1 · 2026-10-17 修订（更新记录）

1. 概述

接口共 6 个，都需要 API 密钥。完成一次兑换按顺序调用其中 4 个；`GET /ping` 用于联调，`POST /orders/lookup` 用于找回订单。

方法与路径	用途	卡密
`GET /ping`	验证密钥，查看来源 IP 和限额	不涉及
`POST /cdk/query`	查询卡密的套餐和状态	不消耗
`POST /redeem/verify`	核对买家账号，取得核对票据	不消耗
`POST /redeem`	提交兑换，取得订单号	锁定
`GET /orders/{order_no}`	查询订单状态和结果	不涉及
`POST /orders/lookup`	按卡密找回订单	不涉及

另附 OpenAPI 3.0 描述文件 `openapi.json`，可以导入 Apifox、Postman 等工具，其中的字段、取值和错误码与本文档一致。

2. 兑换流程

1 **查询卡密（可选）**
买家粘贴卡密后调用 `POST /cdk/query`，得到套餐和卡密状态。不消耗卡密。

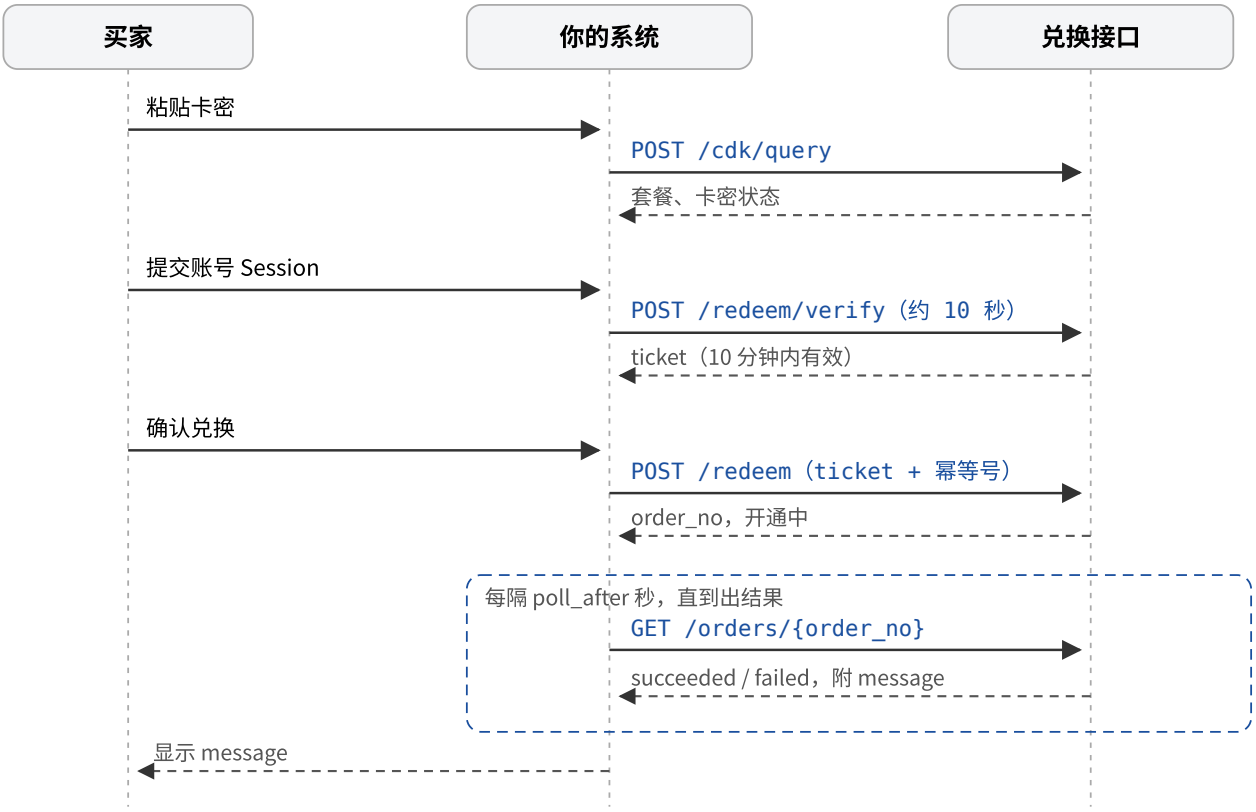
2 **核对账号**
买家提交 ChatGPT 账号的 Session 后调用 `POST /redeem/verify`。耗时约 10 秒，通过后得到核对票据 `ticket`，10 分钟内有效。

3 **提交兑换**

买家确认后调用 `POST /redeem`，传入 `ticket` 和你生成的幂等号 `idempotency_key`，得到订单号 `order_no`。卡密从这一步开始锁定。

4 **查询结果**

按返回的 `poll_after` 秒数轮询 `GET /orders/{order_no}`，直到 `status` 为 `succeeded`（已开通）或 `failed`（失败），通常需要 1–3 分钟。



一次兑换的请求顺序。实线是请求，虚线是返回。

注意

请求超时或没有收到响应时，用同一个 `idempotency_key` 和同一张 `ticket` 重发，或者调用 `POST /orders/lookup` 找回订单，不会重复兑换。不要换新的幂等号，也不要重新核对后再提交。完整规则见 重试规则。

3. 接入准备

开通时我们会提供以下信息：

项目	说明

接口地址	`https://recharge.chongplus.store/api/open/v1`，下文写作 `{BASE}`。
API 密钥	`cpa_` 开头，共 47 个字符。只在创建时显示一次，遗失后需要停用并重新申请。
来源 IP	你的服务器出口 IP 或 IP 段。登记后，这把密钥只接受来自这些地址的请求。调用 `GET /ping` 可以看到接口识别到的来源 IP；请求因来源 IP 被拒绝时，`ip_not_allowed` 的 `error.details.ip` 也是这个地址。

警告

- 密钥只保存在服务器端（环境变量或密钥管理服务），不要写进网页、App 或任何前端代码，也不要发到聊天群或工单里。
- 接口只接受服务器之间的调用，不支持浏览器直接请求（不放行跨域请求）。
- 只使用 `https://`。如果曾经用 `http://` 发送过密钥，按已泄露处理：联系我们停用并更换。
- 买家的 Session 等同于账号的登录凭证，只在本次请求中使用，不要保存，不要写进日志。
- 怀疑密钥泄露时，联系我们停用，停用立即生效。
- 用错误的密钥或从未登记的 IP 反复请求，来源 IP 会被临时封锁 10 分钟。

使用约定

使用本接口即表示同意以下约定：

- 本接口只提供兑换能力，买家使用的页面、文案和样式由你自行设计制作。
- 不得复制、仿制或套用我们网站的页面、样式、图片、视频和教程；不得用 iframe 嵌入我们的网站，也不得用自己的域名转发（镜像）我们的网站。
- 失败的订单只能由买家本人发起重试，你的程序不得自动重新提交，原因见 失败后重新提交。
- 密钥只供你自己的系统使用，不得转借或转卖。
- 订单结果以本接口的返回为准。
- 违反以上任何一条，我们将停用密钥。

4. 通用约定

请求

- HTTPS，JSON（UTF-8）。POST 请求带 `Content-Type: application/json`，请求体不超过 128 KB。
- 认证：每个请求带 `Authorization: Bearer <API 密钥>`。
- `User-Agent` 填写你自己系统的名称，例如 `my-shop/1.0`。部分语言和库的默认值会被网关拒绝，例如 Python 的 urllib 和 pycurl、Java 8 及更早版本、Perl 的 LWP。被拒绝时返回 HTTP 403，响应只有一行纯文本 `error code: 1010`（不是 JSON），请求没有到达接口。
- 超时时间设为 60 秒以上，核对账号一步约需 10 秒。

响应

所有响应都是 JSON。成功时 HTTP 200，`ok` 为 `true`：

JSON

```
{
  "ok": true,
  "data": { ... },
  "trace_id": "9F2A41C07B"
}
```

失败时 `ok` 为 `false`，HTTP 状态码见下表：

JSON

```
{
  "ok": false,
  "error": {
    "code": "cdk_used",
    "message": "卡密已兑换或已使用",
    "retryable": false,
    "cdk_state": "used",
    "details": { "order_no": "OD3F9A1C07B2D4E5" }
  },
  "trace_id": "9F2A41C07B"
}
```

字段	类型	说明
`ok`	boolean	请求是否成功。
`data`	object	成功时的返回内容，各接口不同。
`error.code`	string	错误码，取值固定，程序按它判断。全部取值见 错误码。
`error.message`	string	面向买家的中文说明，可以直接展示；需要自定义文案时按 `error.code` 判断。
`error.retryable`	boolean	同样的操作稍后能否再试。具体怎样重试以各接口的说明为准，有的需要重新核对账号。
`error.retry_after`	integer	建议等待的秒数，同时放在 `Retry-After` 响应头中。只在需要等待的错误中出现。
`error.cdk_state`	string	这一步之后卡密的状态，取值见 卡密状态。不一定出现。
`error.details`	object	附加信息，不一定出现：`order_no` 表示卡密已被你这把密钥提交过，可以用 `GET /orders/{order_no}` 查询；`ip` 只在 `ip_not_allowed` 时出现，是接口识别到的来源 IP。
`trace_id`	string	每个响应都有。联系我们排查问题时请提供。

HTTP 状态码

状态码	含义
200	成功（`ok` 为 `true`）。
400	请求格式错误（`bad_request`），或 Session 格式错误（`session_format`）。
401 / 403	密钥无效或已停用（`invalid_key`）/ 来源 IP 不在登记名单中（`ip_not_allowed`）。
404	接口、核对票据或订单不存在。
409	当前状态不允许这个操作，例如卡密已使用、核对已过期、票据已提交过、账号需要等待。
422	请求格式正确，但内容未通过校验，例如卡密无效、Session 已失效。
429	请求过于频繁、同时处理的订单过多或系统繁忙，按 `Retry-After` 等待后重试。
500 / 503	服务暂时无法处理，稍后重试。

限额

- 每把密钥有两个上限：每分钟请求数和同时处理中的订单数，默认 60 次/分钟和 5 单，以 `GET /ping` 返回的 `limits` 为准，需要调整请联系我们。超出时返回 429（`rate_limited` 或 `too_many_inflight`）。
- 高峰期或请求过密时，可能临时返回 429 `busy`，按 `Retry-After` 稍后重试即可，与密钥本身无关。

时间格式

所有时间都是 RFC 3339 格式的 UTC 时间，例如 `2026-10-05T09:00:00Z`。

5. 接口

下文示例中，`{BASE}` 是接口地址，`$KEY` 是你的密钥，卡密用 `XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX` 代替。

GET /ping

`GET` `/ping` 联调

验证密钥是否有效，返回密钥名称、接口识别到的来源 IP 和限额。登记来源 IP 时以这里返回的 `ip` 为准。没有请求体。

响应字段

字段	类型	说明

`name`	string	密钥名称。
`ip`	string	接口识别到的来源 IP。
`server_time`	string	服务器当前时间。
`limits.rate_per_min`	integer	每分钟请求上限。
`limits.max_inflight`	integer	同时处理中的订单上限。

示例

Shell

```
curl {BASE}/ping -H "Authorization: Bearer $KEY" -H "User-Agent: my-shop/1.0"
```

JSON

```
{
  "ok": true,
  "data": {
    "name": "代理甲",
    "ip": "203.0.113.10",
    "server_time": "2026-10-05T09:00:00Z",
    "limits": { "rate_per_min": 60, "max_inflight": 5 }
  },
  "trace_id": "9F2A41C07B"
}
```

可能的错误

`invalid_key`、`ip_not_allowed`、`too_many_failures`、`rate_limited`、`service_disabled`，说明见 错误码。

POST /cdk/query

POST `/cdk/query` 只读 不消耗卡密

查询卡密对应的套餐和当前状态，不需要买家的账号信息。同一张卡密 60 秒内重复查询，可能返回缓存结果。

如果这张卡密已经由你这把密钥提交过订单：订单还在处理时返回 `cdk_not_usable`，已兑换成功时返回 `cdk_used`，两种情况都在 `error.details.order_no` 中给出订单号。

请求字段

字段	类型	必填	说明
`cdk`	string	是	卡密。不区分大小写，首尾空格会被忽略。

响应字段

字段	类型	说明
`plan`	`string`	套餐代号，取值见下表。
`plan_name`	`string`	套餐名称，可以直接展示。
`cdk_state`	`string`	卡密状态，见 卡密状态。

plan	plan_name
`go`	Go
`plus`	Plus
`pro-5x`	Pro 5x
`pro-20x`	Pro 20x
`pro-max`	Pro Max
`other`	ChatGPT 套餐。以后新增的套餐会先使用这个取值，程序不需要因此修改。

示例

Shell

```
curl -X POST {BASE}/cdk/query -H "Authorization: Bearer $KEY" -H "User-Agent: my-shop/1.0" \
  -H "Content-Type: application/json" \
  -d '{"cdk":"XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX"}'
```

JSON

```
{
  "ok": true,
  "data": { "plan": "pro-5x", "plan_name": "Pro 5x", "cdk_state": "available" },
  "trace_id": "9F2A41C07B"
}
```

可能的错误

错误码	场景	处理
`cdk_invalid`	卡密格式不对或不存在	请买家核对卡密。

`cdk_used`	卡密已兑换	`error.details.order_no` 有值时，是你这把密钥提交的订单，查询该订单。
`cdk_not_usable`	暂时不能使用：已停用、已冻结或正在处理	`error.details.order_no` 有值时，是你这把密钥提交的订单仍在处理，查询该订单；否则请买家联系你，必要时联系我们。
`busy` / `service_unavailable` / `try_later`	系统繁忙或暂时不可用	稍后重试。

POST /redeem/verify

`POST` `/redeem/verify` 约 10 秒 不消耗卡密

核对买家的账号能否兑换这张卡密，通过后返回核对票据 `ticket`。这一步约需 10 秒，请把超时设为 60 秒以上。

请求字段

字段	类型	必填	说明
`cdk`	`string`	是	卡密。
`session`	`string / object`	是	买家的整页 Session，可以是字符串，也可以是 JSON 对象。

> **提示**
>
> 获取 Session：让买家在已登录 ChatGPT 的浏览器中打开 `https://chatgpt.com/api/auth/session`，复制页面上的全部内容（以 `{` 开头，包含 `accessToken`）。只复制其中的 token，或者提供邮箱和密码，都会返回 `session_format`。

示例

Shell

```
curl -X POST {BASE}/redeem/verify -H "Authorization: Bearer $KEY" -H "User-Agent: my-shop/1.0" \
  -H "Content-Type: application/json" \
  -d '{"cdk":"XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX","session":"{\"user\":
{...},\"accessToken\":\"...\"}"}'
```

账号可以兑换：

JSON

```
{
  "ok": true,
  "data": {
```

```
    "eligible": true,
    "ticket": "tk_3f9a1c07b2d4e5a6b7c8d9e0f1a2b3c4",
    "expires_at": "2026-10-05T09:10:00Z",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "account": {
      "email": "bu***@example.com",
      "current_plan": "free",
      "current_plan_name": "Free",
      "has_active_plan": false
    }
  },
  "trace_id": "9F2A41C07B"
}
```


账号已有生效中的套餐，不能叠加。此时 HTTP 仍为 200，但 `eligible` 为 `false`，没有 `ticket`：

JSON


```
{
  "ok": true,
  "data": {
    "eligible": false,
    "reason": "account_has_plan",
    "message": "这个账号已有生效中的套餐，不能叠加，请换一个账号。卡密未消耗",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "account": {
      "email": "bu***@example.com",
      "current_plan": "plus",
      "current_plan_name": "Plus",
      "has_active_plan": true
    }
  },
  "trace_id": "9F2A41C07B"
}
```


响应字段

字段	类型	说明
`eligible`	boolean	为 `true` 时才有 `ticket`。为 `false` 时请买家换一个账号，卡密没有消耗。
`ticket`	string	核对票据，在 `expires_at` 之前有效（10 分钟），只能用于一次兑换。过期后重新核对。
`expires_at`	string	票据的过期时间。
`reason`	string	只在 `eligible` 为 `false` 时出现，目前只有 `account_has_plan`。
`message`	string	只在 `eligible` 为 `false` 时出现，面向买家的说明，可以直接展示。

`plan` / `plan_name`	string	卡密对应的套餐，取值见 POST /cdk/query。
`account.email`	string	打码后的账号邮箱，供买家确认账号。
`account.current_plan`	string	账号当前的套餐：`free`，或者 `plan` 的取值之一。
`account.current_plan_name`	string	账号当前套餐的名称。
`account.has_active_plan`	boolean	账号当前是否有生效中的套餐。为 `true` 时 `eligible` 为 `false`。

可能的错误

错误码	场景	处理
`session_format`	Session 不是整页内容，缺少 `accessToken` 等	请买家按上面的方法重新复制。
`session_invalid`	账号登录已失效	请买家重新登录后再复制一次。
`cdk_invalid` / `cdk_used` / `cdk_not_usable`	卡密的问题	同 POST /cdk/query。
`wait_before_retry`	这个账号刚失败过，需要等待，响应带 `retry_after`	等待 `retry_after` 秒后重新核对。
`account_not_eligible`	这个账号当前不能充值这个套餐	请买家换一个账号。
`request_rejected`	请求未通过	请买家核对卡密和账号后重试。
`busy` / `service_unavailable` / `try_later`	系统繁忙或暂时不可用	稍后重新核对。

POST /redeem

`POST` `/redeem` 幂等 锁定卡密

用核对票据提交兑换。接口立即返回订单，开通在后台进行。卡密从这一步开始锁定。

请求字段

字段	类型	必填	说明
`ticket`	string	是	`POST /redeem/verify` 返回的票据。
`idempotency_key`	string	是	幂等号，8–64 位，可以使用字母、数字和 `. _ : -`。每次新的兑换使用新值，建议用你系统中的订单号或 UUID；同一次兑换的所有重试使用同一个值。

示例

Shell

```
curl -X POST {BASE}/redeem -H "Authorization: Bearer $KEY" -H "User-Agent: my-shop/1.0" \
  -H "Content-Type: application/json" \
  -d '{"ticket":"tk_3f9a1c07b2d4e5a6b7c8d9e0f1a2b3c4","idempotency_key":"order-20261005-000123"}'
```

JSON


```
{
  "ok": true,
  "data": {
    "order_no": "OD3F9A1C07B2D4E5",
    "status": "processing",
    "cdk_state": "held",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "buyer_email": "bu***@example.com",
    "message": "开通中，通常 1–3 分钟。请不要重复提交。卡密已锁定。",
    "created_at": "2026-10-05T09:02:11Z",
    "updated_at": "2026-10-05T09:02:11Z",
    "poll_after": 8
  },
  "trace_id": "9F2A41C07B"
}
```


返回的是订单对象，字段说明见 GET /orders/{order_no}。

重试规则

情况	处理
请求超时、连接中断、没有收到响应，或返回 5xx	用同一个 `idempotency_key` 和同一张 `ticket` 重发，返回的是同一张订单，不会重复兑换。也可以调用 `POST /orders/lookup` 按卡密找回订单。不要换新的幂等号，也不要重新核对后再提交。
返回 `busy`、`too_many_inflight`、`service_unavailable`、`try_later`	这次提交没有执行，票据仍然有效。等待 `retry_after` 秒（没有时等几秒）后，用同一张票据和同一个幂等号重发。
返回其他错误，例如 `wait_before_retry`、`ticket_expired`、`session_invalid`、`account_has_plan`、`cdk_used`、`cdk_not_usable`、`request_rejected`	票据已失效。先处理问题（等待、换账号或重新复制 Session），再从 `POST /redeem/verify` 重新开始，并使用新的幂等号。
返回 `ticket_used`	这张票据已经提交过，`error.details.order_no` 是对应的订单，直接查询该订单。换新的幂等号也不会产生新订单。
返回 `idempotency_key_reused`	这个幂等号已经和另一张票据一起用过，换一个新的。

GET /orders/{order_no}

`GET` `/orders/{order_no}` 轮询

查询订单的当前状态。`order_no` 是 `POST /redeem` 返回的订单号。只能查询你这把密钥提交的订单，其他订单一律返回 404。

响应字段

字段	类型	说明
`order_no`	string	订单号。
`status`	string	订单状态：`processing`、`succeeded`、`failed`、`under_review`，见 订单状态与失败处理。
`cdk_state`	string	卡密当前的状态：`available`、`held`、`used`。
`plan` / `plan_name`	string	套餐，取值见 POST /cdk/query。
`buyer_email`	string	打码后的买家账号邮箱。
`message`	string	订单当前情况，面向买家的中文说明，末尾说明卡密是否消耗。
`created_at` / `updated_at`	string	提交时间 / 最近更新时间。
`finished_at`	string	结束时间，只在 `succeeded`、`failed` 时出现。
`failure.code` / `failure.action`	string	只在 `failed` 时出现：失败原因和建议的处理方式，见 失败原因。
`poll_after`	integer	只在订单未结束时出现，建议多少秒后再查询（开通中 8 秒，人工核对 60 秒）。同一张订单 6 秒内重复查询，得到的是同样的结果。

示例

Shell

```
curl {BASE}/orders/OD3F9A1C07B2D4E5 -H "Authorization: Bearer $KEY" -H "User-Agent: my-shop/1.0"
```

已开通：

JSON

```
{
  "ok": true,
  "data": {
    "order_no": "OD3F9A1C07B2D4E5",
```

```
    "status": "succeeded",
    "cdk_state": "used",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "buyer_email": "bu***@example.com",
    "message": "已开通。卡密已消耗。",
    "created_at": "2026-10-05T09:02:11Z",
    "updated_at": "2026-10-05T09:03:42Z",
    "finished_at": "2026-10-05T09:03:42Z"
  },
  "trace_id": "9F2A41C07B"
}
```


失败：

JSON


```
{
  "ok": true,
  "data": {
    "order_no": "OD3F9A1C07B2D4E5",
    "status": "failed",
    "cdk_state": "available",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "buyer_email": "bu***@example.com",
    "message": "这次付款被风控拦截了，请过两分钟之后再尝试。卡密未消耗。",
    "created_at": "2026-10-05T09:02:11Z",
    "updated_at": "2026-10-05T09:03:42Z",
    "finished_at": "2026-10-05T09:03:42Z",
    "failure": { "code": "payment_blocked", "action": "retry_later" }
  },
  "trace_id": "9F2A41C07B"
}
```


可能的错误

`no_such_order`：订单号不存在，或者不是你这把密钥提交的订单。

POST /orders/lookup

POST `/orders/lookup` 找回订单

按卡密找回订单，用于订单号丢失、页面刷新或提交时没有收到响应的情况。返回你这把密钥对这张卡密提交的最近一张订单，格式与 `GET /orders/{order_no}` 相同；没有时返回 404 `no_such_order`。其他代理提交的订单、买家在我们网站上兑换的订单都查不到。

请求字段

字段	类型	必填	说明

`cdk`	`string`	是	卡密。放在请求体中而不是网址中，避免出现在访问日志里。

示例

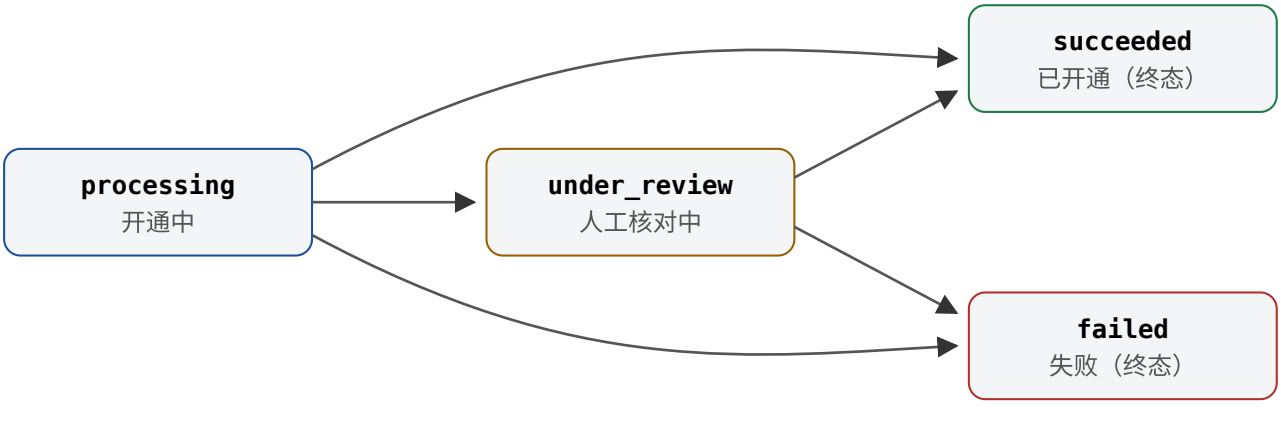
Shell

```
curl -X POST {BASE}/orders/lookup -H "Authorization: Bearer $KEY" -H "User-Agent: my-shop/1.0" \
  -H "Content-Type: application/json" \
  -d '{"cdk":"XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX"}'
```

6. 订单状态与失败处理

订单状态

status	含义	处理
`processing`	开通中，通常 1–3 分钟	按 `poll_after` 继续查询，不要重复提交。卡密已锁定。
`succeeded`	已开通（终态）	提示买家到 ChatGPT 中确认。卡密已消耗。
`failed`	失败（终态）	查看 `failure` 和 `cdk_state`：`cdk_state` 为 `available` 时可以重新开始，其他情况不要重新提交，联系我们。
`under_review`	人工核对中，通常 2 小时内有结果	不要重复提交，按 `poll_after`（60 秒）继续查询，之后会变为 `succeeded` 或 `failed`。需要加急时联系我们并提供订单号。卡密已锁定。



订单从 `processing` 开始，结束为 `succeeded` 或 `failed`；少数订单会先进入 `under_review`。

卡密状态

cdk_state	含义

`available`	未消耗：卡密仍可使用，可以重新核对、重新提交。
`held`	已锁定：正在处理或等待人工核对，不能重复提交。
`used`	已使用：已兑换成功。

失败订单的 `cdk_state` 以最近一次查询为准，失败后卡密恢复为 `available` 可能需要几秒。

失败原因

订单为 `failed` 时，`failure.code` 是失败原因，`failure.action` 是建议界面如何引导买家：

failure.code	failure.action	含义
`session_invalid`	`recopy_session`	账号登录已失效：请买家重新复制 Session，再重新核对。
`account_has_plan`	`change_account`	账号已有生效中的套餐：请买家换一个账号。
`account_restricted`	`change_account`	账号暂时无法充值：过两分钟再试，或者换一个账号。
`payment_blocked`	`retry_later`	付款被风控拦截：过两分钟再试。
`payment_failed`	`retry_later`	付款没有成功：过两分钟再试。
`try_later`	`retry_later`	通道暂时繁忙：过两分钟再试。
`cancelled`	`retry_later`	这次充值已取消。
`not_accepted`	`retry_later`	这次提交没有被受理。
`recharge_failed`	`retry_later`	其他原因未完成。

失败后重新提交

订单为 `failed` 且 `cdk_state` 为 `available` 时，卡密仍可使用。按 `failure.action` 引导买家（等待两分钟、换一个账号或重新复制 Session），再从 `POST /redeem/verify` 重新开始，并使用新的幂等号。部分失败之后（例如付款被风控拦截），同一个账号需要等待约 2 分钟才能再次提交，否则会返回 `wait_before_retry`，响应带 `retry_after`。

`cdk_state` 不是 `available` 时，不要重新提交，联系我们处理。

> **警告**
>
> `failure.action` 为 `retry_later` 时，表示请买家过两分钟后自己再试一次，不是让你的系统过两分钟后自动提交。每次提交都会实际发起一次付款，同一个账号连续失败时，自动重交只会继续失败。重新提交必须由买家本人发起，并从 `POST /redeem/verify` 开始。

7. 错误码

`retryable` 列对应响应中的 `error.retryable`。

error.code	HTTP	retryable	含义	处理
`invalid_key`	401	否	密钥无效或已停用	检查密钥；已停用的联系我们。
`ip_not_allowed`	403	否	来源 IP 不在这把密钥登记的名单中	用 `GET /ping` 查看来源 IP，联系我们登记。
`service_disabled`	503	是	接口暂时关闭	稍后重试。
`rate_limited`	429	是	请求过于频繁	按 `Retry-After` 等待。
`too_many_failures`	429	是	这个 IP 的错误请求过多，已被临时封锁	等待 `Retry-After`，检查密钥和来源 IP。
`busy`	429	是	系统繁忙	按 `Retry-After` 稍后重试。
`too_many_inflight`	429	是	同时处理中的订单已达上限	等已有订单出结果后再提交。
`bad_request`	400	否	请求格式错误	检查请求体。
`not_found`	404	否	接口不存在	检查路径和请求方法。
`cdk_invalid`	422	否	卡密无效	请买家核对卡密。
`cdk_used`	409	否	卡密已兑换或已使用	查看 `error.details.order_no`。
`cdk_not_usable`	409	否	卡密暂时不能使用：已停用、已冻结或正在处理	查看 `error.details.order_no`；没有时联系我们。
`session_format`	400	否	Session 不是整页内容	请买家重新复制整页 Session。
`session_invalid`	422	否	账号登录已失效	请买家重新复制 Session。
`account_not_eligible`	409	否	账号当前不能充值这个套餐	请买家换一个账号。
`account_has_plan`	409	否	账号已有生效中的套餐	请买家换一个账号。
`wait_before_retry`	409	是	这个账号需要等待后再试	等待 `retry_after` 秒后重新核对。
`no_such_ticket`	404	否	核对票据不存在	重新核对账号。
`ticket_expired`	409	否	核对已过期或已作废	重新核对账号。

`ticket_used`	409	否	这张票据已经提交过	查看 `error.details.order_no`。
`idempotency_key_reused`	409	否	幂等号已用于另一张票据	换一个新的幂等号。
`no_such_order`	404	否	订单不存在	检查订单号；只能查询本密钥提交的订单。
`request_rejected`	422	否	请求未通过	请买家核对卡密和账号后重试。
`try_later`	503	是	请过两分钟之后再尝试	稍后重试。
`service_unavailable`	503	是	服务暂时不可用	稍后重试。
`internal_error`	500	是	服务内部错误	稍后重试；持续出现时提供 `trace_id` 联系我们。

来源 IP 不在登记名单中时，响应会给出接口识别到的来源 IP，按这个地址登记即可：

JSON

```
{
  "ok": false,
  "error": {
    "code": "ip_not_allowed",
    "message": "当前请求来源的 IP 不在这把密钥允许的名单里",
    "retryable": false,
    "details": { "ip": "203.0.113.99" }
  },
  "trace_id": "9F2A41C07B"
}
```

8. 示例代码

四份示例完成同一件事：查询卡密、核对账号、提交兑换（按 重试规则 处理超时和未执行的提交）、按 `poll_after` 查询到出结果。只使用语言自带的库（PHP 使用 curl 扩展），设置 `AGENT_API_KEY` 环境变量后即可运行。

提示

- 接入自己的系统时，`idempotency_key` 要在调用 `POST /redeem` 之前和你自己的订单一起写入数据库，进程重启后的重试也使用它。
- 示例用 sleep 轮询是为了便于阅读。实际系统中一般由后台任务按 `poll_after` 定时查询，买家页面只显示当前的 `message`。
- 买家的 Session 只在本次请求中使用，不要保存，不要写进日志。

Python 3.8+，只使用标准库

redeem.py

```
"""兑换接口 v1 示例（Python 3.8+，只用标准库）。

用法:
    export AGENT_API_KEY=cpa_...          # 密钥只放在服务器的环境变量里
    python redeem.py <卡密> <买家 Session 文件>

把这份代码接进你的系统时：idempotency_key 要在调用 /redeem 之前和你自己的订单一起存进数据库，
重试时原样复用；买家的 Session 只在这一次请求里用，不要保存、不要写日志。
"""

import http.client
import json
import os
import secrets
import sys
import time
import urllib.error
import urllib.request

BASE = os.environ.get("AGENT_API_BASE", "https://recharge.chongplus.store/api/open/v1")
KEY = os.environ["AGENT_API_KEY"]
# 换成你自己系统的名字。必须设置：urllib 的默认值（Python-urllib/3.x）会被网关拒绝（HTTP 403，回包是 error code: 1010）。
USER_AGENT = "my-shop/1.0"

# 这几种错误说明「这次没处理」：等 retry_after 秒后，用同一张 ticket、同一个 idempotency_key 原样再发。
RESEND_SAME = {"busy", "rate_limited", "too_many_inflight", "service_unavailable",
               "service_disabled", "try_later", "internal_error"}
UNFINISHED = {"processing", "under_review"}


class ApiError(Exception):
    """接口明确返回了错误（ok=false）。"""

    def __init__(self, status, error, trace_id):
        super().__init__(f"{error.get('code')}: {error.get('message')} (trace_id={trace_id})")
        self.status = status
        self.code = error.get("code", "")
        self.message = error.get("message", "")
        self.retry_after = error.get("retry_after") or 0
        self.details = error.get("details") or {}


class NetworkError(Exception):
    """没拿到响应（超时、断网、网关错误页）：结果未知，只能原样重发或按卡密找回。"""


def call(method, path, body=None):
    data = None if body is None else json.dumps(body, ensure_ascii=False).encode("utf-8")
    req = urllib.request.Request(BASE + path, data=data, method=method)
    req.add_header("Authorization", "Bearer " + KEY)
    req.add_header("User-Agent", USER_AGENT)
    if data is not None:
        req.add_header("Content-Type", "application/json")
    try:
        with urllib.request.urlopen(req, timeout=90) as resp:  # 核对账号一步要 10 秒左右
            status, raw = resp.status, resp.read()
```

```
    except urllib.error.HTTPError as e:  # 4xx / 5xx 也带着 JSON
        status, raw = e.code, e.read()
    except (OSError, http.client.HTTPException) as e:  # 超时、断网、连接被断开
        raise NetworkError(str(e)) from e
    try:
        payload = json.loads(raw)
    except ValueError as e:
        raise NetworkError(f"HTTP {status}，回包不是 JSON") from e
    if payload.get("ok"):
        return payload["data"]
    raise ApiError(status, payload.get("error") or {}, payload.get("trace_id"))


def submit(ticket, idem, cdk):
    """确认兑换。网络出错、或者是「这次没处理」的错误，都用同一张 ticket、同一个幂等号原样再发。"""
    for _ in range(6):
        try:
            return call("POST", "/redeem", {"ticket": ticket, "idempotency_key": idem})
        except NetworkError:
            time.sleep(3)
        except ApiError as e:
            if e.code == "ticket_used" and e.details.get("order_no"):
                return call("GET", "/orders/" + e.details["order_no"])  # 这张 ticket 已经提交过：查那张单
            if e.code not in RESEND_SAME:
                raise  # 其他错误：这张 ticket 作废，处理完问题后从核对账号重新来，并换新的幂等号
            time.sleep(e.retry_after or 3)
    # 一直没拿到明确结果：按卡密找回（可能已经受理了），不要换新的幂等号重新提交。
    return call("POST", "/orders/lookup", {"cdk": cdk})


def wait_until_done(order, deadline_seconds=2 * 3600):
    deadline = time.monotonic() + deadline_seconds
    while order["status"] in UNFINISHED:
        if time.monotonic() > deadline:
            break  # 等太久了：提示买家联系客服并给出 order_no，不要重新提交
        time.sleep(order.get("poll_after") or 8)
        try:
            order = call("GET", "/orders/" + order["order_no"])
        except NetworkError:
            time.sleep(10)
        except ApiError as e:
            if e.code not in RESEND_SAME:
                raise
            time.sleep(e.retry_after or 10)
    return order


def redeem(cdk, session):
    info = call("POST", "/cdk/query", {"cdk": cdk})  # 可选：先告诉买家是什么套餐
    print("套餐：", info["plan_name"], file=sys.stderr)

    check = call("POST", "/redeem/verify", {"cdk": cdk, "session": session})
    if not check["eligible"]:
        return {"ok": False, "message": check["message"]}  # 账号已有套餐：请买家换一个账号

    idem = "ord-" + secrets.token_hex(12)  # 实际使用时：和你的订单一起先存库，重试时原样复用
    order = wait_until_done(submit(check["ticket"], idem, cdk))
    return {
```

```
        "ok": order["status"] == "succeeded",
        "status": order["status"],
        "message": order["message"],
        "order_no": order["order_no"],
        "cdk_state": order["cdk_state"],
        "failure": order.get("failure"),
    }


if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit("用法: python redeem.py <卡密> <买家 Session 文件>")
    with open(sys.argv[2], encoding="utf-8") as f:
        buyer_session = f.read()
    try:
        result = redeem(sys.argv[1], buyer_session)
    except ApiError as e:
        result = {"ok": False, "code": e.code, "message": e.message, "details": e.details}
    print(json.dumps(result, ensure_ascii=False, indent=2))
```

Node.js 18+，不需要安装依赖

redeem.mjs

```
// 兑换接口 v1 示例（Node.js 18+，不用装任何依赖）。
//
// 用法:
//   export AGENT_API_KEY=cpa_...          # 密钥只放在服务器的环境变量里
//   node redeem.mjs <卡密> <买家 Session 文件>
//
// 把这份代码接进你的系统时：idempotency_key 要在调用 /redeem 之前和你自己的订单一起存进数据库，
// 重试时原样复用；买家的 Session 只在这一次请求里用，不要保存、不要写日志。

import { randomBytes } from 'node:crypto'
import { readFileSync } from 'node:fs'
import { setTimeout as sleep } from 'node:timers/promises'

const BASE = process.env.AGENT_API_BASE || 'https://recharge.chongplus.store/api/open/v1'
const KEY = process.env.AGENT_API_KEY
// 换成你自己系统的名字。别用语言自带的默认值：有的会被网关直接拒绝（HTTP 403，回包是 error code: 1010）。
const USER_AGENT = 'my-shop/1.0'

// 这几种错误说明「这次没处理」：等 retry_after 秒后，用同一张 ticket、同一个 idempotency_key 原样再发。
const RESEND_SAME = new Set([
  'busy',
  'rate_limited',
  'too_many_inflight',
  'service_unavailable',
  'service_disabled',
  'try_later',
  'internal_error',
])
const UNFINISHED = new Set(['processing', 'under_review'])

/** 接口明确返回了错误（ok=false）。 */
class ApiError extends Error {
```

```
  constructor(status, error, traceId) {
    super(`${error.code}: ${error.message} (trace_id=${traceId})`)
    this.status = status
    this.code = error.code || ''
    this.userMessage = error.message || '' // 给买家看的那句话
    this.retryAfter = error.retry_after || 0
    this.details = error.details || {}
  }
}

/** 没拿到响应（超时、断网、网关错误页）：结果未知，只能原样重发或按卡密找回。 */
class NetworkError extends Error {}

async function call(method, path, body) {
  let res
  try {
    res = await fetch(BASE + path, {
      method,
      headers: {
        Authorization: `Bearer ${KEY}`,
        'User-Agent': USER_AGENT,
        ...(body === undefined ? {} : { 'Content-Type': 'application/json' }),
      },
      body: body === undefined ? undefined : JSON.stringify(body),
      signal: AbortSignal.timeout(90_000), // 核对账号一步要 10 秒左右
    })
  } catch (e) {
    throw new NetworkError(String(e))
  }
  let payload
  try {
    payload = await res.json()
  } catch {
    throw new NetworkError(`HTTP ${res.status}，回包不是 JSON`)
  }
  if (payload.ok) return payload.data
  throw new ApiError(res.status, payload.error || {}, payload.trace_id)
}

/** 确认兑换。网络出错、或者是「这次没处理」的错误，都用同一张 ticket、同一个幂等号原样再发。 */
async function submit(ticket, idem, cdk) {
  for (let i = 0; i < 6; i++) {
    try {
      return await call('POST', '/redeem', { ticket, idempotency_key: idem })
    } catch (e) {
      if (e instanceof NetworkError) {
        await sleep(3000)
        continue
      }
      if (e.code === 'ticket_used' && e.details.order_no) {
        return call('GET', `/orders/${e.details.order_no}`) // 这张 ticket 已经提交过：查那张单
      }
      if (!RESEND_SAME.has(e.code)) throw e // 其他错误：这张 ticket 作废，处理完问题后从核对账号重新来，并换新的幂
等号
      await sleep((e.retryAfter || 3) * 1000)
    }
  }
  // 一直没拿到明确结果：按卡密找回（可能已经受理了），不要换新的幂等号重新提交。
```

```
  return call('POST', '/orders/lookup', { cdk })
}

async function waitUntilDone(order, deadlineMs = 2 * 3600_000) {
  const deadline = Date.now() + deadlineMs
  while (UNFINISHED.has(order.status)) {
    if (Date.now() > deadline) break // 等太久了：提示买家联系客服并给出 order_no，不要重新提交
    await sleep((order.poll_after || 8) * 1000)
    try {
      order = await call('GET', `/orders/${order.order_no}`)
    } catch (e) {
      if (!(e instanceof NetworkError) && !RESEND_SAME.has(e.code)) throw e
      await sleep((e.retryAfter || 10) * 1000)
    }
  }
  return order
}

export async function redeem(cdk, session) {
  const info = await call('POST', '/cdk/query', { cdk }) // 可选：先告诉买家是什么套餐
  console.error('套餐：', info.plan_name)

  const check = await call('POST', '/redeem/verify', { cdk, session })
  if (!check.eligible) return { ok: false, message: check.message } // 账号已有套餐：请买家换一个账号

  const idem = `ord-${randomBytes(12).toString('hex')}` // 实际使用时：和你的订单一起先存库，重试时原样复用
  const order = await waitUntilDone(await submit(check.ticket, idem, cdk))
  return {
    ok: order.status === 'succeeded',
    status: order.status,
    message: order.message,
    order_no: order.order_no,
    cdk_state: order.cdk_state,
    failure: order.failure,
  }
}

if (process.argv.length === 4) {
  const [cdk, sessionFile] = process.argv.slice(2)
  try {
    console.log(JSON.stringify(await redeem(cdk, readFileSync(sessionFile, 'utf8')), null, 2))
  } catch (e) {
    if (!(e instanceof ApiError)) throw e
    console.log(JSON.stringify({ ok: false, code: e.code, message: e.userMessage, details: e.details },
null, 2))
  }
} else {
  console.error('用法：node redeem.mjs <卡密> <买家 Session 文件>')
  process.exitCode = 1
}
```

Go 1.21+，只使用标准库

redeem.go

```go
// 兑换接口 v1 示例（Go 1.21+，只用标准库）。
//
// 用法：
//
//  export AGENT_API_KEY=cpa_...          # 密钥只放在服务器的环境变量里
//  go run redeem.go <卡密> <买家 Session 文件>
//
// 把这份代码接进你的系统时：idempotency_key 要在调用 /redeem 之前和你自己的订单一起存进数据库，
// 重试时原样复用；买家的 Session 只在这一次请求里用，不要保存、不要写日志。
package main

import (
    "bytes"
    "crypto/rand"
    "encoding/hex"
    "encoding/json"
    "errors"
    "fmt"
    "net/http"
    "os"
    "time"
)

var (
    base   = envOr("AGENT_API_BASE", "https://recharge.chongplus.store/api/open/v1")
    key    = os.Getenv("AGENT_API_KEY")
    client = &http.Client{Timeout: 90 * time.Second} // 核对账号一步要 10 秒左右
)

// 换成你自己系统的名字。别用语言自带的默认值：有的会被网关直接拒绝（HTTP 403，回包是 error code: 1010）。
const userAgent = "my-shop/1.0"

// 这几种错误说明「这次没处理」：等 retry_after 秒后，用同一张 ticket、同一个 idempotency_key 原样再发。
var resendSame = map[string]bool{
    "busy": true, "rate_limited": true, "too_many_inflight": true, "service_unavailable": true,
    "service_disabled": true, "try_later": true, "internal_error": true,
}

// APIError 接口明确返回了错误（ok=false）。
type APIError struct {
    Status     int               `json:"-"`
    Code       string            `json:"code"`
    Message    string            `json:"message"`
    RetryAfter int               `json:"retry_after"`
    Details    map[string]string `json:"details"`
    TraceID    string            `json:"-"`
}

func (e *APIError) Error() string {
    return fmt.Sprintf("%s: %s (trace_id=%s)", e.Code, e.Message, e.TraceID)
}

// ErrNetwork 没拿到响应（超时、断网、网关错误页）：结果未知，只能原样重发或按卡密找回。
var ErrNetwork = errors.New("network error")

// Order 一张订单（只列了这里用到的字段）。
type Order struct {
```

```
    OrderNo   string `json:"order_no"`
    Status    string `json:"status"`
    CDKState  string `json:"cdk_state"`
    Message   string `json:"message"`
    PollAfter int    `json:"poll_after"`
    Failure   *struct {
        Code   string `json:"code"`
        Action string `json:"action"`
    } `json:"failure,omitempty"`
}

func call(method, path string, body, out any) error {
    var reader *bytes.Reader
    if body != nil {
        raw, err := json.Marshal(body)
        if err != nil {
            return err
        }
        reader = bytes.NewReader(raw)
    } else {
        reader = bytes.NewReader(nil)
    }
    req, err := http.NewRequest(method, base+path, reader)
    if err != nil {
        return err
    }
    req.Header.Set("Authorization", "Bearer "+key)
    req.Header.Set("User-Agent", userAgent)
    if body != nil {
        req.Header.Set("Content-Type", "application/json")
    }
    resp, err := client.Do(req)
    if err != nil {
        return fmt.Errorf("%w: %v", ErrNetwork, err)
    }
    defer resp.Body.Close()
    var payload struct {
        OK      bool            `json:"ok"`
        Data    json.RawMessage `json:"data"`
        Error   *APIError       `json:"error"`
        TraceID string          `json:"trace_id"`
    }
    if err := json.NewDecoder(resp.Body).Decode(&payload); err != nil {
        return fmt.Errorf("%w: HTTP %d，回包不是 JSON", ErrNetwork, resp.StatusCode)
    }
    if !payload.OK {
        e := payload.Error
        if e == nil {
            e = &APIError{Code: "internal_error"}
        }
        e.Status, e.TraceID = resp.StatusCode, payload.TraceID
        return e
    }
    if out == nil {
        return nil
    }
    return json.Unmarshal(payload.Data, out)
}
```

```
// submit 确认兑换。网络出错、或者是「这次没处理」的错误，都用同一张 ticket、同一个幂等号原样再发。
func submit(ticket, idem, cdk string) (*Order, error) {
    for i := 0; i < 6; i++ {
        var order Order
        err := call(http.MethodPost, "/redeem", map[string]string{"ticket": ticket, "idempotency_key":
idem}, &order)
        if err == nil {
            return &order, nil
        }
        var apiErr *APIError
        switch {
        case errors.Is(err, ErrNetwork):
            time.Sleep(3 * time.Second)
        case errors.As(err, &apiErr) && apiErr.Code == "ticket_used" && apiErr.Details["order_no"] != "":
            // 这张 ticket 已经提交过：查那张单
            err = call(http.MethodGet, "/orders/"+apiErr.Details["order_no"], nil, &order)
            return &order, err
        case errors.As(err, &apiErr) && resendSame[apiErr.Code]:
            time.Sleep(time.Duration(max(apiErr.RetryAfter, 3)) * time.Second)
        default:
            return nil, err // 其他错误：这张 ticket 作废，处理完问题后从核对账号重新来，并换新的幂等号
        }
    }
    // 一直没拿到明确结果：按卡密找回（可能已经受理了），不要换新的幂等号重新提交。
    var order Order
    err := call(http.MethodPost, "/orders/lookup", map[string]string{"cdk": cdk}, &order)
    return &order, err
}

func waitUntilDone(order *Order, deadline time.Duration) (*Order, error) {
    stop := time.Now().Add(deadline)
    for order.Status == "processing" || order.Status == "under_review" {
        if time.Now().After(stop) {
            break // 等太久了：提示买家联系客服并给出 order_no，不要重新提交
        }
        time.Sleep(time.Duration(max(order.PollAfter, 1)) * time.Second)
        var next Order
        err := call(http.MethodGet, "/orders/"+order.OrderNo, nil, &next)
        var apiErr *APIError
        switch {
        case err == nil:
            order = &next
        case errors.Is(err, ErrNetwork):
            time.Sleep(10 * time.Second)
        case errors.As(err, &apiErr) && resendSame[apiErr.Code]:
            time.Sleep(time.Duration(max(apiErr.RetryAfter, 10)) * time.Second)
        default:
            return order, err
        }
    }
    return order, nil
}

func redeem(cdk, session string) (map[string]any, error) {
    var info struct {
        PlanName string `json:"plan_name"`
    }
```

```
    if err := call(http.MethodPost, "/cdk/query", map[string]string{"cdk": cdk}, &info); err != nil {
        return nil, err
    }
    fmt.Fprintln(os.Stderr, "套餐: ", info.PlanName) // 可选：先告诉买家是什么套餐

    var check struct {
        Eligible bool   `json:"eligible"`
        Ticket   string `json:"ticket"`
        Message  string `json:"message"`
    }
    if err := call(http.MethodPost, "/redeem/verify", map[string]string{"cdk": cdk, "session": session},
&check); err != nil {
        return nil, err
    }
    if !check.Eligible {
        return map[string]any{"ok": false, "message": check.Message}, nil // 账号已有套餐：请买家换一个账号
    }

    buf := make([]byte, 12)
    if _, err := rand.Read(buf); err != nil {
        return nil, err
    }
    idem := "ord-" + hex.EncodeToString(buf) // 实际使用时：和你的订单一起先存库，重试时原样复用
    order, err := submit(check.Ticket, idem, cdk)
    if err != nil {
        return nil, err
    }
    if order, err = waitUntilDone(order, 2*time.Hour); err != nil {
        return nil, err
    }
    return map[string]any{
        "ok": order.Status == "succeeded", "status": order.Status, "message": order.Message,
        "order_no": order.OrderNo, "cdk_state": order.CDKState, "failure": order.Failure,
    }, nil
}

func envOr(name, fallback string) string {
    if v := os.Getenv(name); v != "" {
        return v
    }
    return fallback
}

func main() {
    if len(os.Args) != 3 {
        fmt.Fprintln(os.Stderr, "用法: go run redeem.go <卡密> <买家 Session 文件>")
        os.Exit(1)
    }
    session, err := os.ReadFile(os.Args[2])
    if err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
    result, err := redeem(os.Args[1], string(session))
    var apiErr *APIError
    if errors.As(err, &apiErr) {
        result = map[string]any{"ok": false, "code": apiErr.Code, "message": apiErr.Message, "details":
apiErr.Details}
```

```
    } else if err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
    out, _ := json.MarshalIndent(result, "", "  ")
    fmt.Println(string(out))
}
```

PHP 7.4+，需要 curl 扩展

redeem.php

```
<?php
// 兑换接口 v1 示例（PHP 7.4+，需要 curl 扩展）。
//
// 用法：
//   export AGENT_API_KEY=cpa_...          # 密钥只放在服务器的环境变量里
//   php redeem.php <卡密> <买家 Session 文件>
//
// 把这份代码接进你的系统时：idempotency_key 要在调用 /redeem 之前和你自己的订单一起存进数据库，
// 重试时原样复用；买家的 Session 只在这一次请求里用，不要保存、不要写日志。

define('API_BASE', getenv('AGENT_API_BASE') ?: 'https://recharge.chongplus.store/api/open/v1');
define('API_KEY', (string) getenv('AGENT_API_KEY'));
// 换成你自己系统的名字。别用框架或库自带的默认值：有的会被网关直接拒绝（HTTP 403，回包是 error code: 1010）。
const USER_AGENT = 'my-shop/1.0';

// 这几种错误说明「这次没处理」：等 retry_after 秒后，用同一张 ticket、同一个 idempotency_key 原样再发。
const RESEND_SAME = ['busy', 'rate_limited', 'too_many_inflight', 'service_unavailable',
'service_disabled', 'try_later', 'internal_error'];
const UNFINISHED = ['processing', 'under_review'];

/** 接口明确返回了错误（ok=false）。 */
class ApiError extends Exception
{
    public $status;
    public $errorCode;
    public $userMessage; // 给买家看的那句话
    public $retryAfter;
    public $details;

    public function __construct(int $status, array $error, string $traceId)
    {
        $this->status = $status;
        $this->errorCode = (string) ($error['code'] ?? '');
        $this->userMessage = (string) ($error['message'] ?? '');
        $this->retryAfter = (int) ($error['retry_after'] ?? 0);
        $this->details = is_array($error['details'] ?? null) ? $error['details'] : [];
        parent::__construct("{$this->errorCode}: {$this->userMessage} (trace_id={$traceId})");
    }
}

/** 没拿到响应（超时、断网、网关错误页）：结果未知，只能原样重发或按卡密找回。 */
class NetworkError extends Exception
{
}
```

```php
function call(string $method, string $path, ?array $body = null): array
{
    $headers = ['Authorization: Bearer ' . API_KEY];
    $options = [
        CURLOPT_CUSTOMREQUEST => $method,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_USERAGENT => USER_AGENT,
        CURLOPT_CONNECTTIMEOUT => 10,
        CURLOPT_TIMEOUT => 90, // 核对账号一步要 10 秒左右
    ];
    if ($body !== null) {
        $options[CURLOPT_POSTFIELDS] = json_encode($body, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES
| JSON_THROW_ON_ERROR);
        $headers[] = 'Content-Type: application/json';
    }
    $options[CURLOPT_HTTPHEADER] = $headers;
    $ch = curl_init(API_BASE . $path);
    curl_setopt_array($ch, $options);
    $raw = curl_exec($ch);
    $status = (int) curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
    if ($raw === false) {
        throw new NetworkError(curl_error($ch)); // 超时、断网、连接被断开
    }
    $payload = json_decode($raw, true);
    if (!is_array($payload)) {
        throw new NetworkError("HTTP {$status}，回包不是 JSON");
    }
    if (!empty($payload['ok'])) {
        return $payload['data'];
    }
    throw new ApiError($status, is_array($payload['error'] ?? null) ? $payload['error'] : [], (string)
($payload['trace_id'] ?? ''));
}

/** 确认兑换。网络出错、或者是「这次没处理」的错误，都用同一张 ticket、同一个幂等号原样再发。 */
function submit(string $ticket, string $idem, string $cdk): array
{
    for ($i = 0; $i < 6; $i++) {
        try {
            return call('POST', '/redeem', ['ticket' => $ticket, 'idempotency_key' => $idem]);
        } catch (NetworkError $e) {
            sleep(3);
        } catch (ApiError $e) {
            if ($e->errorCode === 'ticket_used' && !empty($e->details['order_no'])) {
                return call('GET', '/orders/' . rawurlencode($e->details['order_no'])); // 这张 ticket 已
经提交过：查那张单
            }
            if (!in_array($e->errorCode, RESEND_SAME, true)) {
                throw $e; // 其他错误：这张 ticket 作废，处理完问题后从核对账号重新来，并换新的幂等号
            }
            sleep($e->retryAfter > 0 ? $e->retryAfter : 3);
        }
    }
    // 一直没拿到明确结果：按卡密找回（可能已经受理了），不要换新的幂等号重新提交。
    return call('POST', '/orders/lookup', ['cdk' => $cdk]);
}
```

```php
function waitUntilDone(array $order, int $deadlineSeconds = 7200): array
{
    $deadline = time() + $deadlineSeconds;
    while (in_array($order['status'], UNFINISHED, true)) {
        if (time() > $deadline) {
            break; // 等太久了：提示买家联系客服并给出 order_no，不要重新提交
        }
        sleep(max(1, (int) ($order['poll_after'] ?? 8)));
        try {
            $order = call('GET', '/orders/' . rawurlencode($order['order_no']));
        } catch (NetworkError $e) {
            sleep(10);
        } catch (ApiError $e) {
            if (!in_array($e->errorCode, RESEND_SAME, true)) {
                throw $e;
            }
            sleep($e->retryAfter > 0 ? $e->retryAfter : 10);
        }
    }
    return $order;
}

function redeem(string $cdk, string $session): array
{
    $info = call('POST', '/cdk/query', ['cdk' => $cdk]); // 可选：先告诉买家是什么套餐
    error_log('套餐：' . $info['plan_name']);

    $check = call('POST', '/redeem/verify', ['cdk' => $cdk, 'session' => $session]);
    if (empty($check['eligible'])) {
        return ['ok' => false, 'message' => $check['message']]; // 账号已有套餐：请买家换一个账号
    }

    $idem = 'ord-' . bin2hex(random_bytes(12)); // 实际使用时：和你的订单一起先存库，重试时原样复用
    $order = waitUntilDone(submit($check['ticket'], $idem, $cdk));
    return [
        'ok' => $order['status'] === 'succeeded',
        'status' => $order['status'],
        'message' => $order['message'],
        'order_no' => $order['order_no'],
        'cdk_state' => $order['cdk_state'],
        'failure' => $order['failure'] ?? null,
    ];
}

// 直接用命令行运行时才执行下面这段；被你的系统 include 进去时不会执行。
if (PHP_SAPI === 'cli' && realpath($_SERVER['SCRIPT_FILENAME'] ?? '') === __FILE__) {
    if ($argc !== 3) {
        fwrite(STDERR, "用法：php redeem.php <卡密> <买家 Session 文件>\n");
        exit(1);
    }
    $buyerSession = file_get_contents($argv[2]);
    if ($buyerSession === false) {
        exit(1);
    }
    try {
        $result = redeem($argv[1], $buyerSession);
    } catch (ApiError $e) {
        $result = ['ok' => false, 'code' => $e->errorCode, 'message' => $e->userMessage, 'details' =>
```

```
(object) $e->details];
    }
    echo json_encode($result, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT),
PHP_EOL;
}
```

9. 联调与上线检查

联调步骤

1. 调用 `GET /ping`，确认密钥有效，返回的 `ip` 就是你服务器的出口 IP。
2. 向我们申请一张测试卡密，用 `POST /cdk/query` 查询一次，确认能看到套餐。
3. 准备一个没有生效中套餐的测试账号，完整走一遍核对账号、提交兑换、查询订单，直到看到 `succeeded`。没有沙箱环境，测试兑换会实际开通并用掉这张测试卡密。
4. 验证常见的错误分支：卡密写错（`cdk_invalid`）、Session 只复制了一部分（`session_format`）、同一个幂等号连续提交两次（返回同一张订单）、账号已有套餐（`eligible` 为 `false`）。

上线前检查

- 密钥只在服务器的环境变量里，没有出现在前端代码、日志和聊天记录中。
- `GET /ping` 返回的来源 IP 与登记的出口 IP 一致。
- 所有请求的超时不少于 60 秒，核对账号一步不会因为耗时长被当成失败。
- 请求带的是自己系统的 `User-Agent`，不是语言自带的默认值。
- 一次兑换只生成一个幂等号，所有重试都复用它；超时后不重新核对，也不换幂等号。
- 轮询间隔不短于 `poll_after`，并设有总的等待上限（例如 2 小时）；超过上限时提示买家联系客服，并给出 `order_no`。
- 买家的 Session 不入库、不进日志；你的日志里只记录 `order_no` 和 `trace_id`。
- 买家界面直接显示 `message`，失败时按 `failure.action` 引导买家。
- 订单失败后，只有买家本人点了重试才重新核对、重新提交，程序里没有任何自动重交。
- 收到 429、503 时按 `Retry-After` 等待后再试，不立即重试。
- 联调步骤中的错误分支都已验证，界面上的提示与本文档一致。

10. 常见问题

核对账号为什么需要 10 秒左右?

这一步要实时校验买家的账号，属于正常耗时。请把超时设长，并在界面上提示买家正在核对账号。

买家连续点了两次确认兑换会怎样?

一张票据只能使用一次：同一个幂等号的第二次请求返回同一张订单，换了幂等号的第二次请求返回 `ticket_used`，不会重复兑换。

订单一直是 processing 怎么办?

通常 1–3 分钟出结果。超过 10 分钟时继续按 `poll_after` 查询；超过 2 小时请联系我们并提供 `order_no`。不要重新提交。

买家想用另一个账号兑换同一张卡密?

只要卡密仍是 `available`（核对没有通过，或兑换失败后卡密已恢复），就可以用新的账号重新调用 `POST /redeem/verify`。

收到 403，但响应不是 JSON?

响应是一行 `error code: 1010` 时，请求在网关就被拒绝了，原因是 `User-Agent`（见 通用约定），与密钥和 IP 无关，换成你自己系统的名称即可。密钥或 IP 的问题，响应都是 JSON（`invalid_key`、`ip_not_allowed`）。

失败后能让程序过两分钟自动重新提交吗?

不能。每次提交都会实际发起一次付款，同一个账号连续失败时，自动重交只会继续失败。请把 `message` 显示给买家，由买家决定换一个账号，还是稍后自己再试。

联系我们时需要提供什么?

`order_no`（或响应中的 `trace_id`）、出问题的大致时间、卡密的前 8 位和后 4 位，以及你收到的完整响应（请去掉密钥）。

11. 更新记录

日期	内容
2026-10-17	文档改版：调整章节结构，增加接口一览、订单状态图，字段表补充类型和是否必填，示例代码改为分语言切换；PDF 增加书签。接口本身没有变化。
2026-10-08	新增 PHP 示例；新增使用约定；说明订单失败后不要让程序自动重新提交。接口本身没有变化。
2026-10-05	v1 首版。

兑换接口 v1 · 2026-10-17 修订。随附 `openapi.json` 和示例代码（Python、Node.js、Go、PHP）。接口如有不兼容的变动，我们会提前通知。